

UNIVERSIDAD PANAMERICANA
Facultad De Ingeniería y Ciencias Aplicadas
Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación
Programación V/ Base de datos 2



Proyecto HIPSTAGRAM – Fase 3
(PLAN DE PRUEBAS UNITARIAS)

Integrantes:

Douglas Esaú Catú Otzoy (000140060)
Kevin Eliazar Herrera Alburez (000113637)
Priscila Andrea Güitz del Cid (000139626)
Carlos Enrique Otoniel Bautista Sarazua (000133047)
Herbert David Estrada Trujillo (000141126)

PLAN DE PRUEBAS UNITARIAS

Proyecto HIPSTAGRAM

Plataforma de Red Social con Microservicios

Versión	1.0
Fecha	Junio 2026
Equipo	Equipo de Desarrollo — 5 integrantes
Curso	Proyecto General — Ing. Mario Obed Morales Guitz

Índice

1. Introducción.....	1
1.1 Beneficios esperados.....	1
1.2 Herramientas Utilizadas.....	1
2. Alcance de las pruebas.....	2
2.1 Beneficios esperados.....	2
2.2 Beneficios esperados.....	2
3. Entorno de pruebas	3
3.1 Configuración de infraestructura local.....	3
3.2 Dependencias del sistema de calidad.....	3
3.2 Variables de entorno simuladas	3
4. Estrategias de prueba	4
4.1 Beneficios esperados.....	4
4.2 Casos negativos.....	4
4.3 Casos límite	4
5. Diseño de pruebas.....	5
5.1 Desglose de escenarios	6
6. Implementación técnica	7
6.1 Desglose de escenarios	7
6.2 Desglose del patrón AAA	7
7. Ejecución de pruebas	8
7.1 Salida real de la consola.....	8
7.2 Interpretación de resultados	8

1. Introducción

Las pruebas unitarias son los fragmentos de código automatizados en las que podemos realizar verificaciones de pequeñas partes de un programa y verificar si funciona de manera aislada y correcta, estas son útiles ya que permite detectar errores en las etapas tempranas del desarrollo de un sistema, esete es mucho más rápido y económico para corregir funciones rotas y arreglarlas en el momento antes de que pueda colapsar todo el sistema. Este tipo de pruebas brindan aislamiento ya que no dependen de otros elementos o archivos, se realizan simulaciones para que las pruebas sean rápidas y confiables.

1.1 Beneficios esperados

Diseñar un marco estricto de pruebas unitarias aislado para el Backend de Hipstagram empleando el patrón AAA, con el fin de certificar la integridad de la logica de negocio, el control de excepciones y los mecanismos de validación de datos en los microservicios antes de su integración y el despliegue final.

- Detección temprana de errores: Identificar fallos de sintaxis, excepciones imprevistas o comportamientos no controlados en los controladores antes del despliegue en entornos de staging o producción.
- Aseguramiento de la calidad del código (QA): Validar que las capas de seguridad de la aplicación bloqueen de manera infalible las peticiones maliciosas lo que garantiza el cumplimiento de las reglas de negocio de la plataforma.
- Facilidad de mantenimiento: Funcionará como una red de seguridad automatizada, cualquier cambio futuro en el código fuente podrá ser verificado inmediatamente, mitigando el riesgo de regresiones lógicos.

1.2 Herramientas Utilizadas

- Jest: Utilizado como motor de ejecución, entorno de aserciones y creado de estructuras de ailamiento lógicas.
- Supertest: Bibblioteca de abstracción para agentes HTTP en memoria, permite simular llamadas reales a la API REST de Express sin necesidad de levantar un socket de red físico eb el puerto 3000.
- Mocks de base de datos y Middleware: Mecanismos de intercepción basados en las funciones nativas de Jest, diseñado para sustituir el comportamiento interactivo del pool de concexiones PostgreSQL y los validadores de firma eléctrica de tokens.

2. Alcance de las pruebas

El alcance técnico define con precisión los componentes funcionales bajo examen y las fronteras operativas del entorno de control de calidad.

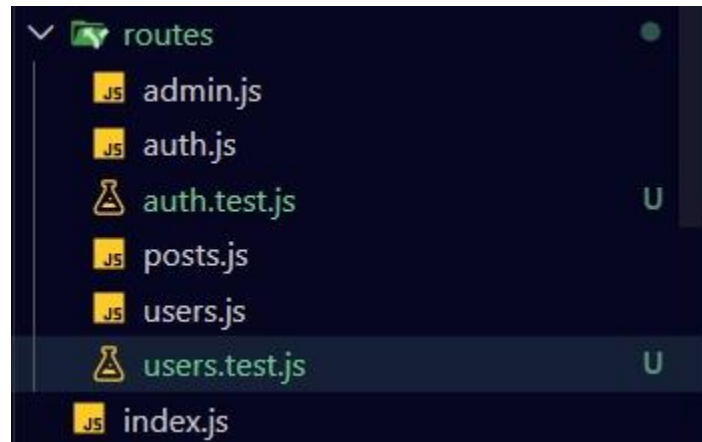
2.1 Beneficios esperados

- Módulo de autenticación: Procesos de registro de cuentas nuevas, validación estructural de enteradas de datos, encriptación de claves, verificación de estados de cuenta lógicos, control de colisiones de correo electrónico, inicio de sesión y renovación cíclica de tokens.
- Módulo de usuarios y administración: Recuperación de perfiles privados, normalización dinámica de URLs multimedia, validación de disponibilidad de usernames, control de acceso basado en roles (RBAC) para funciones administrativas, persistencia volátil de palabras prohibidas, consulta de perfiles externos y protección del árbol de enrutamiento.
- Capa de control estadístico de respuestas: Evaluación de códigos de estado HTTP estándar.

2.2 Beneficios esperados

- Pruebas de interfaz de usuarios: No se evalúa la renderización de elementos en el navegador web ni el comportamiento híbrido en dispositivos móviles a través de capacitor.
- Persistencia real: Queda completamente excluida la mutación física de registros en la base de datos PostgreSQL de AWS y la carga de imágenes en AWS S3.
- Pruebas No funcionales: No se abordan análisis de rendimiento bajo carga cargas de stress, simulación de concurrencia extrema ni auditorías dinámicas de seguridad contra ataques distribuidos en denegación de servicio.

3. Entorno de pruebas



3.1 Configuración de infraestructura local

El entorno para el diseño y ejecución de la suite de software está constituido por las siguientes especificaciones:

- Entorno de ejecución: Node.js
- Framework Web Principal: Express.js
- Editor de código fuente: Visual Studio Code

3.2 Dependencias del sistema de calidad

Las herramientas de análisis se encuentran integradas directamente dentro del manifiesto de desarrollo del proyecto como package.json, requiriendo los siguientes paquetes:

- Jest: Framework de ejecución central.
- supertest: Agente de peticiones HTTP en memoria.
- bcrypt: Manejo simulado de funciones criptográficas y hashing.
- jsonwebtoken y cookie-parser: Manipulación lógica de cabeceras de autorización y extracción de cookies de refresco.

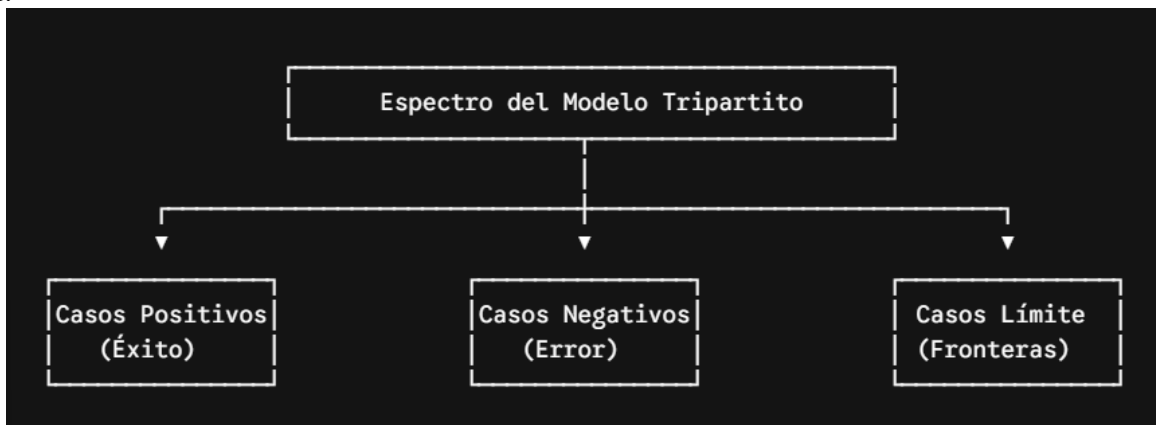
3.3 Variables de entorno simuladas

Para evitar colisiones con el archivo .env global de producción, la suite inyecta forzosamente un contexto volátil al inicio de la ejecución en sus hilos de procesamiento:

- Process.env.JWT_SECRET = 'secret_test'
- Process.env.JWT_REFRESH_SECRET = 'secret_refresh_test'
- Process.env.NODE_ENV = 'development'

4. Estrategias de prueba

La validación estructural se fundamenta en un modelo repartido en tres casos para cubrir todo el espectro de respuesta del backend.



4.1 Beneficios esperados

Evalúan que el servidor responde de manera idónea cuando los datos de entrada y las credenciales cumplen estrictamente con los requisitos del sistema, un claro ejemplo representativo es la consulta del perfil privado como es GET /users/me, donde se verifica que el servidor entregue un estado de HTTP y realice correctamente la conversión algorítmica de URLs relativas de imagen.

4.2 Casos negativos

En este apartado se verifica la resiliencia del backend ante solicitudes inválidas o bien ante ataques, entonces las evaluaciones de escenarios donde un usuario común intenta invocar una ruta reservada de moderación administrativa, esperando un bloque de seguridad con código, a la vez se comprueban la denegación de acceso perimetral ante contraseñas incorrectas.

4.3 Casos límite

Evalúan el comportamiento del sistema en condiciones extremas de captura de datos, como lo son las peticiones de registro con cadenas de texto completamente vacías, los strings que no corresponden a estructuras válidas de correo electrónico, los parámetros faltantes en cadenas de consulta en los Query parameters o bien en la intercepción sintáctica de rutas estáticas versus comodines dinámicos.

5. Diseño de pruebas

La siguiente tabla presenta la matriz de especificación de los 16 casos de prueba que componen las dos suites automatizadas en el proyecto.

auth.test.js

ID	Caso de prueba	Precondiciones	Datos de entrada simulados	Resultado esperado
1	Retornar 400 si los campos obligatorios están ausentes o vacíos.	Servidor express iniciado con router de autenticación	Payload con nombre de usuario, correo y contraseña válidos	Error 400, los campos son obligatorios.
2	Retornar 400 si el correo no es válido.	Enrutamiento perimetral operativo.	Objeto JSON con correo	Error 400, el correo no es válido.
3	Debe retornar 409 si el nombre de usuario no está registrado.	Intercepción del pool de BD, simulación de coincidencia de registros existentes.	Objeto con credenciales válidas completas.	HTTP 400, las credenciales ya están usadas.
4	Debe registrar exitosamente al usuario y configurar la cookie JWT.	Pool simulado en la BD libre de duplicidades y retorno de nueva fila estructurada.	Payload de datos de registro de usuario inédito.	HTTP 201 que presenciu el accessToken y Cookie, así como el refreshToken.
5	Retornar 403 si el usuario existe, pero está inactivo.	BD simulado para que la existencia de un registro este inactivo.	Credenciales del login para una cuenta bloqueada.	HTTP 403, un mensaje de contacto al administrador.
6	Retornar 401 y auditar el login fallido si la contraseña es incorrecta.	BD que simula el usuario que retorna false.	Par de autenticación con contraseña incorrecta.	HTTP 401, mensaje de error de credenciales que llama a la auditoría LOGIN_FALLIDO.
7	Retornar 401 si no se envía la cookie del refresh token	Controlador de renovación configurado con lector de cookies.	Solicitud POST dirigida a /auth/refresh sin cabeceras de cookies.	HTTP 401, desautorizado.
8	Debe limpiar la cookie y dar 401 si el token es inválido.	Validar JWT	Solicitud HTTP con Cookie	HTTP Status 401, envío de cabecera con limpieza de token.

5.1 Desglose de escenarios

Users.test.js

ID	Caso de prueba	Precondiciones	Datos de entrada simulados	Resultado esperado
9	Debe recuperar la información del perfil y normalizar URLs relativos de posts	Middleware de autenticación inyectando sesión válida, Pool simulando perfil y posts mixtos.	Petición GET dirigida al endpoint	HTTP 200, conversión del path local a absoluto mediante prefijo web.
10	Retornar 409 si el nombre de usuario ya está ocupado.	Retornar 409 si el nombre de usuario ya está ocupado.	Parámetro de consulta de URL	HTTP 409, respuesta estructural estricta, available: false.
11	Debe retornar 400 si falta el parámetro username.	Endpoint operativo bajo lectura de query string.	Solicitud GET dirigida a /users/check omitiendo parámetros.	HTTP 400., JSON: { error: 'Falta username.' }.
12	Debe denegar el acceso a /admin/posts si el rol es común.	Variable global mockContexto inicializada bajo la propiedad rol: 'USER'.	Petición GET dirigida a la ruta restringida /users/admin/posts.	HTTP 403, mensaje de error indicando acceso denegado por rol insuficiente.
13	PATCH /users/admin/posts/:id/:action - Debe aprobar posts y registrar auditoría.	Escalado dinámico en memoria a rol: 'ADMIN'. Pool simulando fila afectada en UPDATE.	Invocación de ruta paramétrica /users/admin/posts/500/approve.	HTTP 200, confirmación de aprobación. Sentencia SQL conteniendo la actualización de estado a 'PUBLICADO'. Ejecución de pool.audit con metadatos de red unificados.
14	PUT /users/admin/banned-words - Debe actualizar la lista volátil en memoria.	Contexto configurado con privilegios elevados.	Objeto JSON con estructura estructurada: { banned: ['scam', 'malware'] }.	HTTP 200, el cuerpo de la respuesta debe reflejar la equivalencia exacta de la matriz mutada.
15	Debe retornar 404 si el usuario externo no existe o está inactivo.	BD interceptada retornando un arreglo de filas vacío.	Petición GET con ID paramétrico inexistente en la ruta /users/.	HTTP 404, JSON conteniendo el mensaje { error: 'Usuario no encontrado.' }
16	Frontera Paramétrica: /check no debe ser interceptado por /:id.	interceptado por /:idÁrbol de Express configurado con precedencia estática. Pool simulando respuesta libre.	Llamada a la URL estática /users/check?username=test_user.	HTTP Status: 200. Garantía estructural de que el enrutador procesó la ruta fija y no ejecutó la consulta SQL selectiva de la ruta paramétrica.

6. Implementación técnica

En el análisis del comportamiento del código fuente bajo el paradigma de desarrollo guiado por aserciones utilizando la metodología Arrange, Act y Assert.

6.1 Desglose de escenarios

Para anular las conexiones del backend a la infraestructura real, se configura las intercepciones perimetrales, la implementación del control dinámico de roles en `users.test.js` de la siguiente manera:

```
let mockContexto = { sub: 45, rol: 'USER', ip: '::ffff:127.0.0.1' };

jest.mock('../middleware/auth', () => ({
  requireAuth: (req, res, next) => {
    req.user = { sub: mockContexto.sub, rol: mockContexto.rol };
    req.ip = mockContexto.ip;
    next();
  },
  requireRole: (roleExigido) => (req, res, next) => {
    if (mockContexto.rol !== roleExigido) {
      return res.status(403).json({ error: 'Acceso denegado. Rol insuficiente.' });
    }
    next();
  }
}));
```

6.2 Desglose del patrón AAA

Para ilustrar la implementación del patrón de diseño AAA, se analiza el bloque de código encargado de la moderación y auditoria administrativa de publicaciones:

```
test('PATCH /users/admin/posts/:id:action - Debe aprobar posts y registrar auditoria', async () => {
  mockContexto.rol = 'ADMIN';
  poolModule.query.mockResolvedValueOnce({ rowCount: 1 });

  const response = await request(app).patch('/users/admin/posts/500/approve');

  expect(response.status).toBe(200);
  expect(response.body.message).toContain('aprobado');

  expect(poolModule.query).toHaveBeenCalledWith(
    expect.stringContaining('UPDATE publicacion SET estado=$1'),
    ['PUBLICADO', '500']
  );

  expect(poolModule.audit).toHaveBeenCalledWith(
    mockContexto.sub,
    'ADMIN_APPROVE',
    'publicacion',
    { post_id: '500' },
    mockContexto.ip
  );
});
```

7. Ejecución de pruebas

El comando de activación para inicializar el ciclo de calidad se realiza a través de la terminal integrada del IDE de código, navegando a la raíz del directorio del servidor y ejecutando el script unificado:

npm test

automatizada de aseguramiento editor de código, navegando a la

7.1 Salida real de la consola

Al completar el ciclo analítico, el motor jest genera la siguiente estructura en la terminal de Visual Studio Code:

```
PS C:\Users\chaud\OneDrive\Escritorio\U\7mo. Semestre\Progra 5\Hipstagram\Pr
_Kevin\backend> npm test

> backend@1.0.0 test
> jest

PASS scr/routes/auth.test.js
PASS scr/routes/users.test.js

Test Suites: 2 passed, 2 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        3.076 s
Ran all test suites.
```

7.2 Interpretación de resultados

La lectura analítica del reporte de ejecución certifica los siguientes indicadores fundamentales:

1. Pass: Confirma que ninguna de las aserciones lógicas, comparaciones de tipos de estados HTTP configurada dentro de los archivos arrojó una discrepancia frente al comportamiento real del controlador.
2. Test Suites: Indica el aislamiento e integridad absoluta de los dos archivos de pruebas analizados de forma independiente.
3. Time: Demuestra la alta eficiencia temporal lograda mediante la anulación de llamadas de red de sockets físicos. Procesar 16 peticiones de integración HTTP complejas en poco más de 3 segundos certifica la idoneidad del aislamiento arquitectónico mediante mocks.